

Retrieval-Augmented Generation: Technical Overview

Sample Document for RAG PDF Chatbot Demo

1. What is Retrieval-Augmented Generation?

Retrieval-Augmented Generation (RAG) is an AI architecture that combines information retrieval with large language model (LLM) generation. Instead of relying solely on the knowledge baked into the LLM during training, RAG first fetches relevant documents from an external knowledge base and then uses those documents as grounding context for the model answer. This approach significantly reduces hallucination and keeps answers factual and up to date.

Retrieval-Augmented Generation: Technical Overview

Sample Document for RAG PDF Chatbot Demo

2. Core Components of a RAG Pipeline

A typical RAG pipeline consists of four stages: (a) Document Ingestion: PDFs, web pages, or other sources are loaded and split into smaller, overlapping text chunks (e.g. 1000 characters with 200-character overlap). (b) Embedding: Each chunk is converted into a dense vector representation using an embedding model such as sentence-transformers/all-MiniLM-L6-v2 (384 dimensions) or OpenAI text-embedding-3-small (1536 dimensions). (c) Vector Store: Embeddings are stored in a vector database such as FAISS, Pinecone, or Chroma, enabling millisecond-speed approximate nearest-neighbour search at query time. (d) Generation: The user query is embedded, the top-k most similar chunks are retrieved, and they are injected into a prompt alongside the query before being sent to the LLM.

Retrieval-Augmented Generation: Technical Overview

Sample Document for RAG PDF Chatbot Demo

3. FAISS: Facebook AI Similarity Search

FAISS is an open-source library developed by Meta AI Research for efficient similarity search over dense vectors. It supports billions of vectors on GPU and millions on CPU. In this project we use the CPU-only faiss-cpu build with the IndexFlatL2 index (exact L2 search). For larger corpora, the IVFFlat or HNSW indices provide an accuracy/speed trade-off. FAISS indices are serialisable to disk and can be reloaded without re-embedding, making them ideal for serverless or Streamlit Cloud deployments.

Retrieval-Augmented Generation: Technical Overview

Sample Document for RAG PDF Chatbot Demo

4. Llama 3.3 70B Versatile: The Generation Model

Llama 3.3 70B Versatile, served through Groq's free API, is the generation model for this project. It offers a 128000-token context window and strong performance on reasoning and long-document tasks, with Groq's inference hardware returning tokens fast enough to stream answers with almost no perceptible delay. For RAG, a low sampling temperature (0.2) is recommended to keep answers grounded and consistent. The model is accessed via the Groq API and integrated into this project through LangChain's ChatGroq wrapper. Query rewriting for follow-up questions uses the smaller, faster Llama 3.1 8B Instant model on the same API.

Retrieval-Augmented Generation: Technical Overview

Sample Document for RAG PDF Chatbot Demo

5. LangChain Integration

LangChain is an open-source framework that provides composable building blocks for LLM applications. In this project it supplies: PyPDFLoader: wraps PyPDF for document loading. RecursiveCharacterTextSplitter: hierarchical text chunking. HuggingFaceEmbeddings: wrapper around sentence-transformers. FAISS vector store wrapper: with save/load and as_retriever(). ChatGroq: chat model wrapper for the Groq API. PromptTemplate: custom prompt construction.

Retrieval-Augmented Generation: Technical Overview

Sample Document for RAG PDF Chatbot Demo

6. Chunking Strategy

Text chunking has a significant impact on RAG quality. A chunk that is too large may exceed the embedding model token limit and dilute the semantic signal. A chunk that is too small may lack the necessary surrounding context. The RecursiveCharacterTextSplitter used in this project uses a separator hierarchy: paragraph breaks, line breaks, sentence endings, and finally individual characters. Chunk size is set to 1000 characters with an overlap of 200 to prevent information loss at chunk boundaries.

Retrieval-Augmented Generation: Technical Overview

Sample Document for RAG PDF Chatbot Demo

7. Evaluation Metrics

Common RAG evaluation metrics include: Faithfulness: does the answer contain only information present in the retrieved context? Answer Relevancy: how closely does the answer address the question? Context Precision: what proportion of retrieved chunks were actually useful? Context Recall: were all relevant chunks retrieved? Frameworks such as RAGAS and TruLens provide automated scoring using an LLM-as-a-judge approach, enabling continuous quality monitoring in production.

Retrieval-Augmented Generation: Technical Overview

Sample Document for RAG PDF Chatbot Demo

8. Deployment on Streamlit Cloud

This application can be deployed for free on Streamlit Community Cloud: 1. Push the repository to GitHub (public or private). 2. Visit share.streamlit.io and connect your GitHub account. 3. Select the repository, branch (main), and entry point (app.py). 4. Add GROQ_API_KEY under Settings, Secrets. 5. Click Deploy, the app will be live at a public URL in under two minutes. Note: Streamlit Cloud free tier does not persist the filesystem between sessions, so the vectorstore/ directory is rebuilt on each new session. For persistence, use a cloud object store such as AWS S3 or GCS to cache the FAISS index.